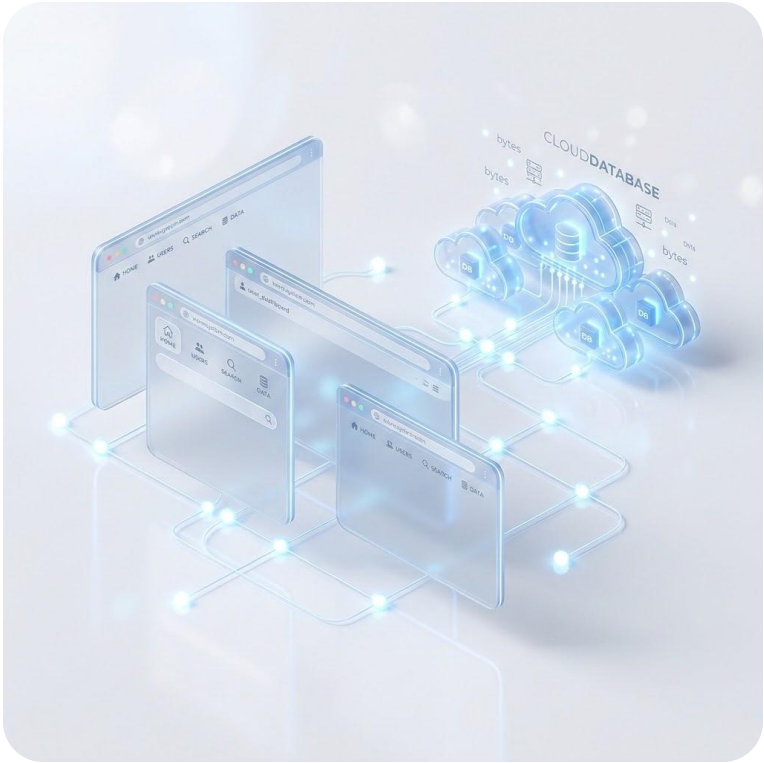
Departamento de Informática
UFV Universidade Federal de Viçosa

INF 321

Projeto e Desenvolvimento de Sistemas para a Web

Unidade 1: Introdução à Web

Prof. Lucas Vegi



Conteúdo

01 **Breve histórico**
- Conceito e surgimento
- Linha do tempo

02 **Modelo cliente-servidor**

03 **Protocolo HTTP**

04 **Arquitetura em camadas**

05 **Padrão Model-View-Controller**

O que é a Internet?

Conectividade Global

A **Internet** é uma rede mundial que conecta milhões de computadores e dispositivos, permitindo a troca de informações por meio de protocolos de comunicação.

Atenção!

Internet e Web **não são a mesma coisa**.

A Web é apenas um dos diversos serviços que utilizam a infraestrutura da Internet para funcionar.

Principais Características

Rede de Alcance Mundial
Infraestrutura global e descentralizada.

Comunicação entre Dispositivos
Interconexão de múltiplos sistemas de hardware.

Compartilhamento de Recursos
Distribuição ágil de informações e serviços.

Base para Múltiplas Aplicações
Web, E-mail, Streaming, Jogos, Nuvem, etc.

Como tudo começou

A Origem da Internet

Final da década de **1960**

Durante a Guerra Fria, o governo dos Estados Unidos buscava uma forma de manter a comunicação mesmo após possíveis ataques nucleares.

Contexto Histórico

A necessidade de sobrevivência a falhas de comunicação militar moldou a arquitetura descentralizada e resiliente que hoje conhecemos.

Objetivos do Projeto

Resistência a Falhas

A rede deveria ser tolerante a falhas pontuais de conexão.

Descentralização

Não depender de um único computador ou nó central.

Continuidade Operacional

Continuar funcionando mesmo com a infraestrutura parcial destruída.

A grande inovação

A ideia revolucionária

Em vez de enviar uma mensagem inteira por um único caminho...

Cada pacote de dados pode seguir uma rota totalmente independente para alcançar o destino final.

Pacotes de dados

A informação é fragmentada em unidades autônomas que trafegam de forma dinâmica e inteligente pela infraestrutura da rede.

Principais Vantagens

Maior confiabilidade
Garante a integridade na entrega da mensagem.

Maior disponibilidade
Rede sempre pronta para transmissão de dados.

Tolerância a falhas
Rotas alternativas em caso de queda de nós.

Comunicação contínua
Fluxo constante e resiliente de informações.

Redes distribuídas

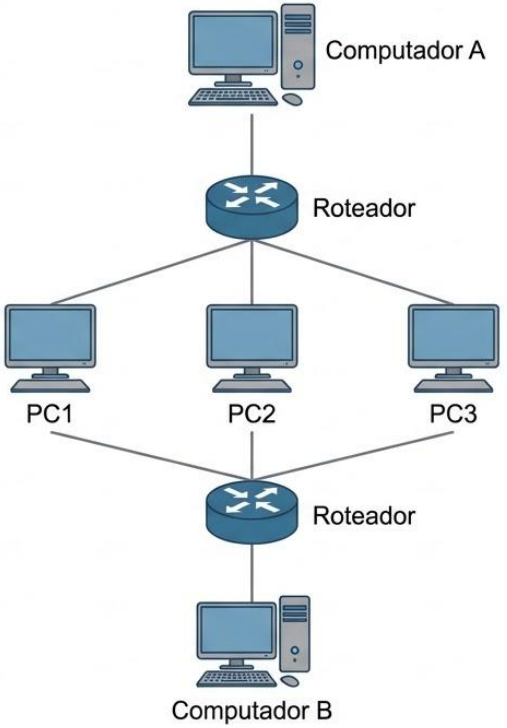
Como funciona uma rede distribuída?

Tolerância a falhas

Se um caminho falhar, os pacotes de dados podem seguir por outro caminho alternativo para chegar ao destino.

Alta robustez

Essa característica de descentralização e redundância torna as redes distribuídas muito mais robustas e confiáveis do que as centralizadas.



O que significa "Internet"?

Origem do nome

Internet

O termo simplificado amplamente conhecido e utilizado hoje em dia.

→

Internetworking

O conceito técnico original de interconexão entre redes de computadores.

→

Interligação de redes

A tradução e significado literal que define a essência da tecnologia.

Conceito Fundamental

A principal ideia era conectar diversas redes independentes para que funcionassem como uma única grande rede mundial.

Principais marcos tecnológicos

A evolução cronológica que moldou a internet moderna

1969 • ARPANET
Primeira rede de computadores interconectada da história.

1971 • E-mail
O surgimento do primeiro grande serviço de comunicação na rede.

1982–1983 • TCP/IP
Criação do protocolo padrão que possibilitou a internet moderna.

1984 • DNS
Implementação do sistema de resolução e criação de nomes de domínio.

1990–1992 • WWW
O nascimento da World Wide Web, revolucionando o acesso à informação.

1993 • Mosaic
Lançamento do primeiro navegador web gráfico de base popular.

1994 • W3C
Início da padronização de tecnologias essenciais como HTML, CSS e XML.

1995 • Java e E-commerce
O surgimento da linguagem Java e o início da internet comercial.

2000+ • Internet Móvel
Conectividade global e crescimento exponencial de múltiplos serviços digitais.

LINHA DO TEMPO DA EVOLUÇÃO DA INTERNET



1957–1969
Origem da Internet

O lançamento do satélite Sputnik motiva os EUA a criarem a ARPA. Em 1969 surge a ARPANET, com os primeiros quatro nós conectando universidades americanas.

- **1957**: Lançamento do satélite Sputnik
- Criação da ARPA (Advanced Research Projects Agency)
- **1969**: Surge a **ARPANET** com os primeiros quatro nós (UCLA, Stanford, UCSB e Universidade de Utah)



1970–1979
Primeiras Tecnologias

Desenvolvimento dos primeiros protocolos de comunicação (NCP), criação do e-mail, divisão do protocolo TCP/IP, surgimento da USENET e dos primeiros sistemas de chat.

- Primeiros protocolos de comunicação (NCP)
- Criação do e-mail (1971)
- Divisão do protocolo TCP/IP (1978)
- USENET – Grupos de Discussão (1979)
- Primeiros sistemas de chat (1979)



1980–1989
Consolidação da Rede

Adoção do TCP/IP como padrão da internet, criação do DNS, crescimento do número de servidores, expansão internacional, surgimento do IRC, primeiro vírus da internet (Internet Worm) e criação do CERT para resposta a incidentes de segurança.

- Adoção do TCP/IP como padrão (1983)
- Criação do DNS (1984)
- Crescimento para dezenas de milhares de servidores
- Expansão internacional
- Surgimento do IRC (1988)
- Internet Worm (1988) e criação do CERT (1988)



1990–1994
Nascimento da Web

Extinção da ARPANET, criação da World Wide Web (WWW) por Tim Berners-Lee, lançamento dos primeiros navegadores (Mosaic e Netscape), surgimento dos primeiros provedores comerciais (ISP), crescimento acelerado da internet e criação do W3C, responsável pela padronização das tecnologias da Web.

- Fim da ARPANET
- Criação da WWW (1990–1992)
- Primeiros navegadores: Mosaic (1993) e Netscape (1994)
- Surgimento dos primeiros ISP comerciais
- Criação do W3C (1994) para padronização da Web



1995–1998
Popularização Comercial

Expansão do comércio eletrônico, lançamento do Java, evolução dos navegadores (Internet Explorer e Netscape), crescimento dos mecanismos de busca, portais, bancos on-line, streaming e XML. A internet torna-se uma plataforma comercial global.

- Comércio eletrônico e bancos on-line
- Lançamento do Java (1995)
- Evolução dos navegadores: Internet Explorer e Netscape
- Crescimento dos mecanismos de busca e portais
- Streaming e XML



2000–Atualidade
Era da Internet Moderna

Expansão da banda larga, internet móvel, redes sociais, computação em nuvem, streaming, Internet das Coisas (IoT) e bilhões de dispositivos conectados, tornando a internet essencial para comunicação, educação, entretenimento e negócios.

- Banda larga e internet móvel
- Redes sociais
- Computação em nuvem
- Streaming de vídeo e música
- Internet das Coisas (IoT) e bilhões de dispositivos conectados

Da ARPANET a uma rede global que conecta o mundo: colaboração, inovação e informação sem fronteiras.

Resumo

Principais conceitos

Visão Geral

Uma síntese dos pilares fundamentais que explicam a origem, a arquitetura de rede e a importância da Internet até os dias atuais.

Origem Histórica
A Internet surgiu por motivos militares e de pesquisa científica.

Resistência a Falhas
Foi projetada desde o início para ser altamente tolerante a falhas.

Envio por Pacotes
Utiliza a comutação de pacotes para a transmissão eficiente de informações.

Etimologia
O termo "Internet" deriva do conceito técnico de *Internetworking*.

Alcance Global
Atualmente conecta bilhões de dispositivos e suporta diversos serviços, incluindo a Web.

Discussão em Grupo

Reflexão e debate



Vocês conseguem citar aplicações que utilizam a Internet, mas que não são a Web?

Pense nos protocolos, serviços e sistemas que operam na infraestrutura da rede de forma independente do ecossistema HTTP/WWW.

Conceituação

Infraestrutura vs. Serviço



Se a internet é infraestrutura, então o que é Web?

Pense na diferença entre a grande rede global de computadores interconectados e o sistema de informação que utiliza essa rede para transferir documentos multimídia.

Internet ≠ Web

Muitas pessoas utilizam os termos como sinônimos, mas eles representam conceitos totalmente diferentes.



Internet

A Infraestrutura • Transporta dados

É a rede física e lógica global de comunicação que conecta os dispositivos.

- Cabos e fibras ópticas
- Satélites e roteadores
- Protocolos de rede (TCP/IP)



Web

O Serviço • Utiliza a infraestrutura

É um sistema de informação que opera sobre a Internet para transferir documentos.

- Páginas e sites web
- Aplicações Web modernas
- Documentos HTML e hipermídia

Analogia

Uma analogia simples para compreender a relação entre a infraestrutura e os serviços.

Imagine uma cidade…

A Internet

As Ruas, Avenidas e Rodovias

Corresponde a toda a infraestrutura física e lógica que permite o tráfego e interconexão global.

- **Sem ruas não existe acesso:** Sem a infraestrutura de rede, é impossível conectar dispositivos ou alcançar qualquer serviço.

A Web

As Lojas, Escolas e Residências

Corresponde aos sistemas, páginas e construções de informação que existem apoiados sobre essas vias.

- **As ruas continuam existindo:** Se removermos os sites e serviços (as construções), a infraestrutura física da rede ainda permanece ativa.

Outros serviços da Internet

A Web é apenas um deles

Além da Web, existem diversos serviços fundamentais que utilizam a infraestrutura da Internet para operar.

E-mail

Permite a troca de mensagens de forma assíncrona globalmente.

Exemplos:

Gmail, Outlook

Streaming

Transmissão contínua e eficiente de áudio e vídeo sob demanda.

Exemplos:

Netflix, Spotify

Videoconf.

Comunicação em tempo real por vídeo, voz e chat interativo.

Exemplos:

Zoom, Teams, Meet

Jogos Online

Entretenimento interativo e multiplayer em redes globais.

Exemplos:

Steam, Xbox Live

Acesso e Arq.

Protocolos para transferência segura de arquivos e controle.

Exemplos:

FTP, SSH

Desafio & Reflexão

O WhatsApp utiliza a Web?

Para responder a essa pergunta, considere a diferença fundamental entre a **Internet** e a **Web**:

- Como o aplicativo do celular se comunica com os servidores?
- O que acontece quando você abre o *WhatsApp Web* no navegador?
- Ele depende do protocolo HTTP ou utiliza outros protocolos da Internet?

Resposta

O WhatsApp utiliza a Web?

Não necessariamente!

Ele utiliza a Internet, mas seu protocolo principal não é HTTP para troca de mensagens.

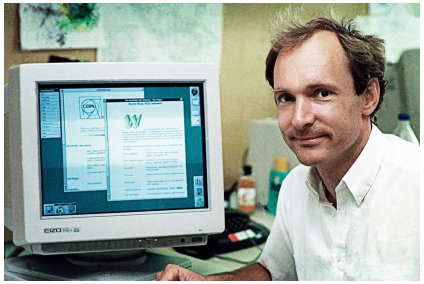
O WhatsApp utiliza o protocolo ***XMPP (eXtensible Messaging and Presence Protocol)*** para gerenciar o sistema de entrega de mensagens.

Como surgiu?

O nascimento da Web

A Origem no CERN

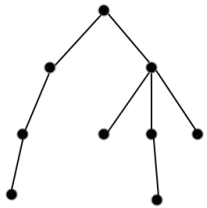
Em **1989-1990**, o pesquisador **Tim Berners-Lee**, do CERN, propôs um sistema para compartilhar documentos científicos entre pesquisadores.



A Ideia Revolucionária

A proposta consistia em um conceito extraordinariamente simples: **documentos contendo links** para outros documentos.

Nascia o conceito de hipertexto.

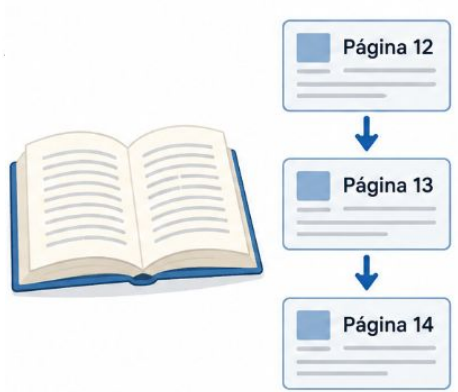


Essa conexão de documentos deu origem à Web como conhecemos hoje.

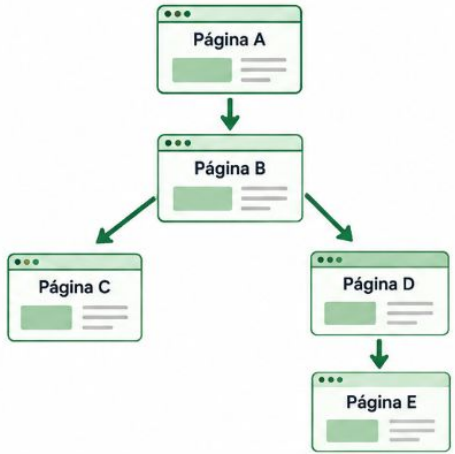
Como funciona?

A ideia revolucionária

Antes da Web: Linear



Na Web: Hipertexto



Agora o usuário escolhe seu próprio caminho.

Os três pilares da Web

A Web foi construída sobre três tecnologias essenciais

Estrutura

HTML

Define e organiza a estrutura de todas as páginas e conteúdos visualizados na Web.

Localização

URL

Representa o endereço único de cada recurso disponível na internet.

`https://www.ufv.br`

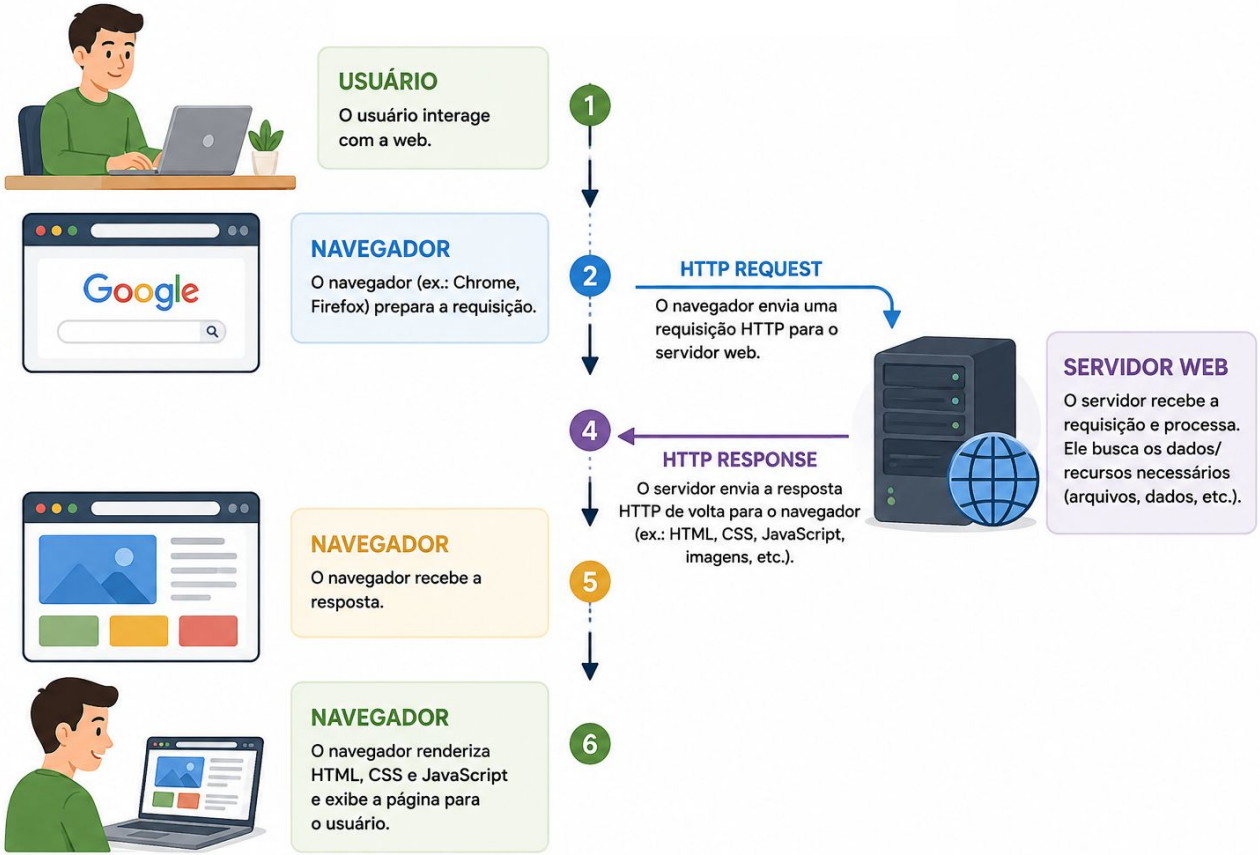
Comunicação

HTTP

O protocolo de transferência que permite a comunicação e troca de dados entre navegador e servidor.

Esses três elementos deram origem à Web moderna.

Fluxo simplificado da Web



O papel do navegador

O browser é a principal ferramenta de interação do usuário com a Web

Browser

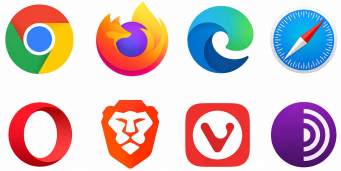
Responsabilidades

- Solicitar páginas
- Interpretar HTML
- Aplicar CSS
- Executar JavaScript
- Exibir o conteúdo ao usuário

Mercado

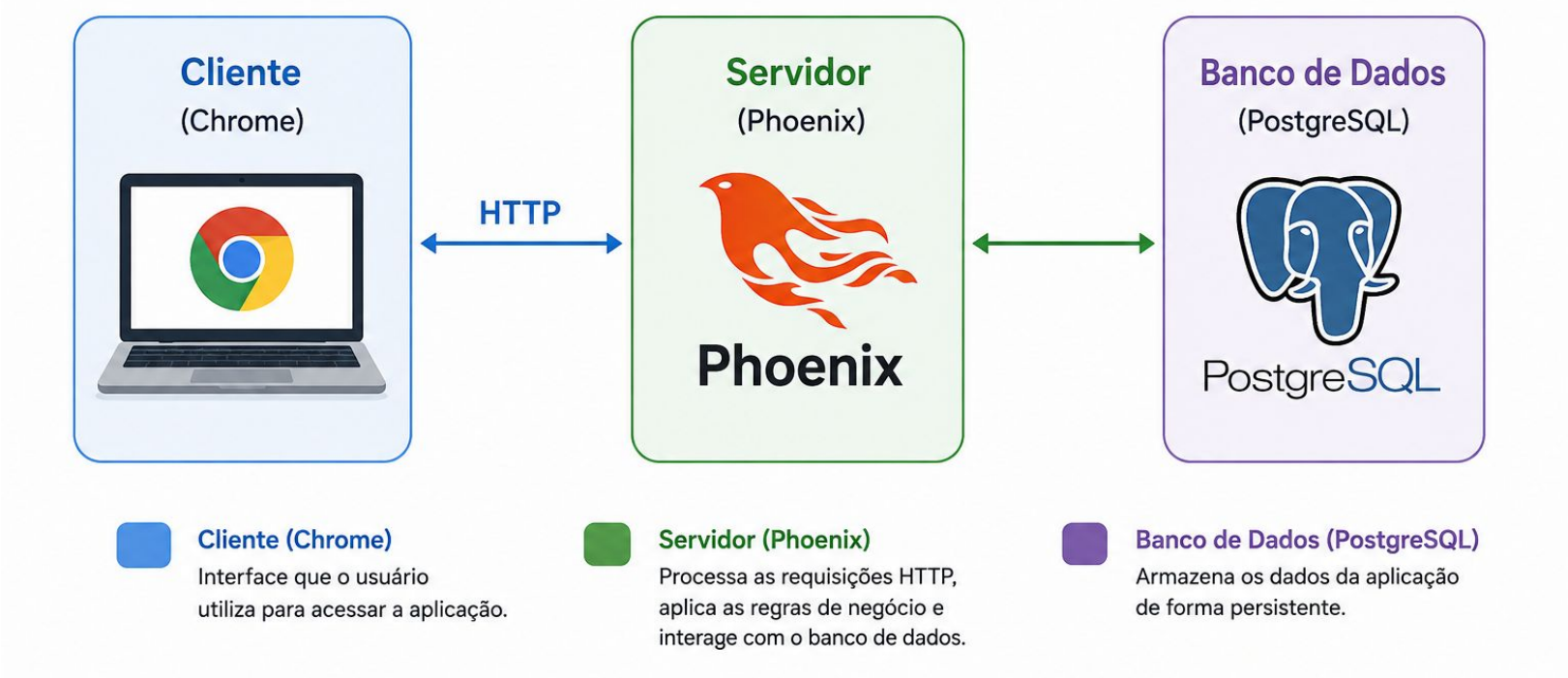
Exemplos

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Apple Safari, etc..



Arquitetura Cliente-Servidor

Na disciplina, construiremos exatamente esse tipo de aplicação utilizando Elixir + Phoenix



O ciclo de uma requisição

O que acontece quando digitamos uma URL?

https://www.google.com

A jornada dos dados

Ao digitar um endereço no navegador, uma série de eventos ordenados é disparada para localizar o servidor e carregar a página.

Este fluxo garante que você acesse qualquer aplicação web de forma rápida, segura e eficiente.

1. Navegador identifica a URL
2. Encontra o endereço IP do servidor (DNS)
3. Estabelece a conexão
4. Envia uma requisição HTTP
5. Servidor processa a requisição
6. Envia uma resposta de volta
7. Navegador renderiza a página

O que forma uma página?

Uma página web é constituída de três tecnologias que trabalham juntas:



Web Estática × Web Dinâmica

Evolução da Web: Como as páginas entregam conteúdo ao navegador

Web Estática

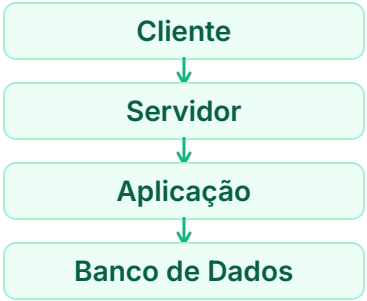
Sempre entrega o mesmo conteúdo para todos os usuários.



O arquivo solicitado é enviado diretamente ao navegador sem processamento prévio.

Web Dinâmica

O conteúdo pode mudar dinamicamente conforme o usuário.



O servidor processa a requisição por meio de aplicações e consultas ao banco para gerar o HTML final.

Onde entra o Phoenix?

O que construiremos na disciplina?

Cliente

Navegador (*e.g.*, Chrome)

Interface com a qual o usuário interage diretamente, disparando requisições e recebendo respostas visuais.

HTTP

Servidor (Phoenix)

Controllers

Recebe e direciona requisições HTTP

Business Logic

Regras de negócio e fluxo da aplicação

Ecto

Interface e integração com o Banco de Dados

SQL

Banco

PostgreSQL

Armazenamento e persistência estruturada dos dados da aplicação.

Resumo

Até aqui aprendemos...

Conceitos-Chave

Uma visão geral sobre os pilares fundamentais da tecnologia que sustenta os sistemas e aplicações web modernos.

01 Internet e Web são conceitos diferentes.

02 A Web foi criada em 1989-1990 por Tim Berners-Lee.

03 A Web utiliza HTML, URLs e HTTP.

04 O navegador comunica-se com servidores por meio de requisições HTTP.

05 Aplicações Web seguem a arquitetura Cliente-Servidor.

HTTP - O protocolo da Web

Principais conceitos e funcionamento da comunicação na Web

01 O que é uma requisição HTTP?

02 O que é uma resposta HTTP?

03 O que são métodos GET, POST, PUT e DELETE?

04 O que são cabeçalhos (*headers*)?

05 O que significam os códigos 200, 404 e 500?

06 Como um navegador conversa com um servidor Phoenix?

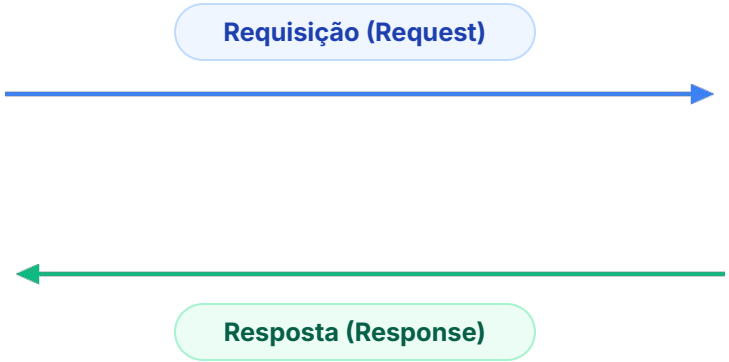
HTTP: Hypertext Transfer Protocol

O protocolo que permite a comunicação na Web

Cliente

Navegador / Browser

Dispara requisições HTTP para interagir com recursos e páginas na Web.



Servidor

Servidor Web

Processa as requisições recebidas e retorna as respostas HTTP adequadas.

Onde estamos?

Relembrando o conteúdo anterior e o que vem a seguir

Aprendemos que:

01 **Internet**
Infraestrutura de comunicação global

02 **Web**
Serviço que utiliza a Internet

03 **HTTP**
Protocolo de comunicação na Web



Agora vamos entender:

Como essa comunicação realmente acontece?

O modelo HTTP

A comunicação funciona principalmente através do modelo de Request e Response

O Cliente

Navegador / Browser

01 Faz uma requisição (*request*)

02 Espera uma resposta (*response*)

O Servidor

Servidor Web

01 Recebe a requisição

02 Processa a requisição interna

03 Retorna uma resposta adequada

Um exemplo simples

Entendendo o fluxo básico de uma requisição e resposta na Web

1. O Navegador Envia *(Request)*

Dados enviados ao acessar https://www.ufv.br

```
GET / HTTP/1.1
Host: www.ufv.br
```

2. O Servidor Responde *(Response)*

Código de status e conteúdo retornado

```
HTTP/1.1 200 OK
Content-Type: text/html
<html>
  ...
</html>
```

3. Renderização: O navegador interpreta o HTML recebido e apresenta a página visual ao usuário.

Uma requisição HTTP

Os principais componentes que estruturam uma mensagem de solicitação

Exemplo Prático

Mensagem enviada pelo navegador

```
GET /produtos HTTP/1.1
Host: exemplo.com
Accept: text/html

[Body / Conteúdo Opcional]
```

01 **Método** `GET`
Define a ação (ex: buscar dados de produtos).

02 **URL / Path** `/produtos`
O endereço ou caminho do recurso solicitado.

03 **Headers** `Host, Accept`
Metadados que trazem informações sobre o cliente.

04 **Body** *(Opcional)*
Dados adicionais enviados (necessário para POST/PUT).

Métodos HTTP

Os métodos indicam a intenção e o objetivo de cada requisição

GET	Obter dados do servidor
POST	Criar / enviar novos dados
PUT	Atualizar / substituir recurso completo
PATCH	Atualizar dados parcialmente
DELETE	Remover recurso do servidor



Uma forma simples de memorizar:

GET → buscar
POST → criar
PUT → substituir
PATCH → alterar
DELETE → remover

O Método GET

Utilizado exclusivamente para solicitar e obter informações do servidor

Exemplos Práticos

Como as requisições são estruturadas

```
GET /users HTTP/1.1
Host: exemplo.com
```

"Servidor, quero obter os usuários."

```
GET /users/42 HTTP/1.1
```

"Quero obter o usuário 42."

Características do GET

Padrões de funcionamento e restrições

Normalmente não possui Body: As requisições servem apenas para leitura. Dados adicionais devem ir na URL ou nos Headers.

Método Seguro e Idempotente: Requisições repetidas retornam o mesmo resultado e não devem alterar o estado do servidor.

O Método POST

Utilizado para enviar informações e criar novos recursos no servidor

Exemplo Prático

Requisição contendo o corpo (Body)

```
POST /users HTTP/1.1
Content-Type: application/json

{
  "name": "Lucas",
  "email": "lucas@example.com"
}
```

Envio de dados ao servidor

O cliente envia informações estruturadas no corpo da mensagem para criar um novo recurso.

Características do POST

Padrões de funcionamento e uso comum

Possui Body: Permite o envio de grandes volumes de dados de forma estruturada no corpo da requisição.

Não Idempotente: Requisições repetidas criam múltiplos recursos diferentes no servidor.

Exemplos de Uso: Criar usuário, cadastrar produto, enviar formulários e realizar pedidos.

PUT e PATCH

Diferentes abordagens para a atualização de recursos no servidor

Método PUT

Normalmente representa a substituição do recurso.

```
PUT /users/42 HTTP/1.1
Content-Type: application/json

{
  "name": "Lucas",
  "email": "novo@email.com"
}
```

Substituição Integral

Envia o objeto completo. Campos omitidos no corpo da requisição podem ser excluídos ou limpos no servidor.

Método PATCH

Utilizado para modificações parciais.

```
PATCH /users/42 HTTP/1.1
Content-Type: application/json

{
  "email": "novo@email.com"


}
```

Modificação Parcial

Altera somente os dados especificados. Os campos não incluídos na mensagem permanecem inalterados.

O Método DELETE

Utilizado para remover de forma permanente recursos armazenados no servidor

Exemplo Prático

Como a requisição de remoção é estruturada

```
DELETE /users/42 HTTP/1.1
Host: exemplo.com
```

Significado da Requisição

O cliente solicita formalmente ao servidor: *"Remova de forma definitiva o usuário que possui o identificador 42."*

Características do DELETE

Padrões de funcionamento e regras de uso

Exclusão de Recursos: Utilizado para remover um recurso. Geralmente não possui ou ignora dados enviados no corpo (Body).

Método Idempotente: Requisições idênticas e repetidas devem produzir o mesmo efeito final (o recurso permanece deletado).

Crucial para CRUD e REST: Mapeia-se diretamente à operação de exclusão (Delete) em APIs e bancos de dados estruturados.

CRUD × HTTP

A relação fundamental para o design e desenvolvimento de APIs REST

Mapeamento de Conceitos

Associação direta de operações à semântica HTTP

Operação CRUD	Método HTTP
Create (Criar)	`POST`
Read (Ler)	`GET`
Update (Atualizar)	`PUT / PATCH`
Delete (Remover)	`DELETE`

Exemplos de Endpoints

Modelagem prática de rotas em recursos

```
POST /products
GET /products
GET /products/10
PATCH /products/10
DELETE /products/10
```

Conexão com APIs REST

Esse padrão de mapeamento de operações para rotas e métodos correspondentes define as regras de arquiteturas RESTful.

Headers

Metadados essenciais enviados junto com as requisições e respostas HTTP

Exemplo de Requisição

Estrutura típica dos headers em uma mensagem

```
GET /users HTTP/1.1
Host: exemplo.com
Accept: application/json
Authorization: Bearer ...
User-Agent: Chrome
```

Os headers carregam metadados que controlam cache, tipo de conteúdo, autenticação e sessão do cliente.

Alguns Headers Importantes

Os campos mais comuns utilizados na Web

- `Host` : Domínio do servidor (ex: exemplo.com)
- `Content-Type` : Formato do corpo da mensagem
- `Accept` : Formatos de mídia aceitáveis na resposta
- `Authorization` : Credenciais para autenticação
- `User-Agent` : Dados do sistema e navegador do cliente
- `Cookie` : Chaves de identificação de sessão ativa

Content-Type

Informa o formato do conteúdo que está sendo transmitido nas mensagens HTTP

Função do Header

Identificação do payload de dados enviado

O header `Content-Type` é o metadado que diz ao destinatário exatamente como interpretar os bytes no corpo (Body) da mensagem.

Aplicações em APIs REST

Isso é crucial ao trabalhar com APIs que trocam dados estruturados, garantindo que o servidor e o cliente saibam processar o formato **JSON** corretamente.

Exemplos Comuns

Os formatos de mídia mais utilizados na Web

```
Content-Type: text/html

Content-Type: application/json

Content-Type:
application/x-www-form-urlencoded
```

text/html : Utilizado para enviar páginas web e documentos HTML estruturados.

application/json : Formato leve de intercâmbio de dados usado em APIs RESTful.

x-www-form-urlencoded : Padrão para envio de dados simples de formulários.

HTTP Response

A estrutura e os componentes essenciais de uma mensagem de retorno do servidor

Exemplo Prático

A anatomia real de uma mensagem de resposta

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "name": "Lucas"
}
```

O exemplo mostra uma resposta bem-sucedida (200 OK) contendo um cabeçalho e um corpo estruturado em JSON.

Elementos da Resposta

Os componentes que formam o retorno

Status Code
Informa o resultado da requisição (ex: HTTP/1.1 200 OK indica que a requisição foi bem-sucedida).

Headers
Metadados que descrevem a mensagem e o servidor (ex: Content-Type indica o formato do corpo).

Body
O conteúdo ou recurso retornado pelo servidor (ex: um documento JSON com os dados solicitados).

Status Codes

Entenda o significado das classes de respostas das requisições HTTP

O que significam os números?

Os códigos de status HTTP possuem três dígitos e são emitidos pelo servidor em resposta a uma requisição do cliente.

`200 OK` = Requisição bem-sucedida

`404 Not Found` = Recurso não encontrado

`500 Server Error` = Erro interno do servidor

Classes de Respostas

A primeira classe define a categoria do código:

`1xx` : Informações adicionais

`2xx` : Sucesso da operação

`3xx` : Redirecionamento de rota

`4xx` : Erro originado no cliente

`5xx` : Erro originado no servidor

Principais códigos

Os códigos de status HTTP mais comuns que você precisa conhecer

2xx — Sucesso

`200 OK`

`201 Created`

`204 No Content`

3xx — Redirecionamento

`301 Moved Permanently`

`302 Found`

`304 Not Modified`

4xx — Erro do cliente

`400 Bad Request`

`401 Unauthorized`

`403 Forbidden`

`404 Not Found`

5xx — Erro do servidor

`500 Internal Server Error`

`502 Bad Gateway`

`503 Service Unavailable`

401 × 403

Uma diferença importante que aparece frequentemente no desenvolvimento Web.

`401` — Unauthorized

O cliente não está autenticado ou não forneceu credenciais válidas.

Analogia:

"Quem é você?"

`403` — Forbidden

O cliente foi identificado, mas não possui permissão para acessar o recurso.

Analogia:

"Eu sei quem você é, mas você não pode fazer isso."

O Body

O corpo contém o conteúdo de dados transmitido de fato na requisição ou resposta.

Requisição *(Request)*

Envia os dados necessários para o servidor

```
POST /users

{
  "name": "Lucas",
  "age": 35
}
```

O formato **JSON** é o mais comum nas APIs modernas para estruturar dados textuais complexos de forma leve.

Resposta *(Response)*

Retorna os dados processados e persistidos

```
HTTP/1.1 201 Created

{
  "id": 42,
  "name": "Lucas",
  "age": 35
}
```

Neste caso, o servidor responde confirmando a criação com sucesso do registro e devolve o novo ID.

HTTP é Stateless

Entenda por que cada requisição Web é independente e como contornamos isso

O que é Stateless?

O servidor **não lembra** do cliente após responder uma requisição.

Cada requisição HTTP é completamente **independente** e isolada das demais.

Simplifica a arquitetura da web, mas exige soluções extras para manter estados.

Fluxo de Requisições

Request 1

`Cliente ⟶ Servidor`

Request 2

`Cliente ⟶ Servidor`

Request 3

`Cliente ⟶ Servidor`

E como fazemos login?

Para manter o estado do usuário ativo, usamos mecanismos auxiliares:

- **Cookies**
- **Sessões**
- **Tokens / JWT**

Estudaremos estes tópicos em detalhes posteriormente.

HTTP e segurança

Entenda a diferença essencial entre a transmissão aberta e a comunicação criptografada

`HTTP` — Inseguro

Os dados trafegam livremente pela rede em texto claro, expostos a qualquer intermediário.

```
Cliente ──[ dados em texto ]──▶ Servidor
```

Vulnerabilidade:

"Os dados podem ser interceptados."

`HTTPS` — Seguro

A conexão estabelece um canal blindado por algoritmos modernos de segurança.

```
Cliente ══[ criptografado ]══▶ Servidor
```

Funcionamento:

HTTPS = HTTP + TLS (Transport Layer Security)

HTTP/1.1, HTTP/2 e HTTP/3

O protocolo também evoluiu

HTTP/1.1

- Muito utilizado
- Comunicação baseada em texto
- Múltiplas requisições podem exigir várias conexões

HTTP/2

- Multiplexação de requisições
- Melhor aproveitamento da conexão
- Compressão de headers

HTTP/3

- Baseado em **QUIC**
- Utiliza UDP como transporte
- Melhora desempenho e resiliência em certas condições

Mas os conceitos fundamentais de Request/Response continuam.

Vamos observar uma requisição

Use o DevTools do navegador para analisar o tráfego de rede em tempo real.

1. Passo a Passo

Abra o painel de desenvolvedor: `F12 → Network`

Acesse o endereço: `https://www.ufv.br`

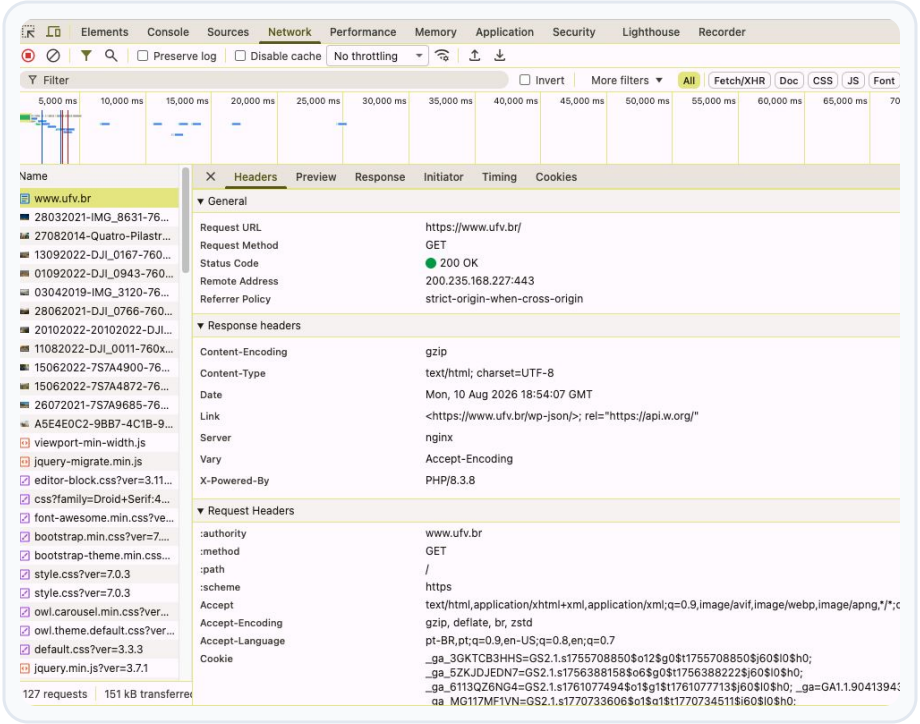
2. O que Observar?

- **Request URL & Method** e **Status Code**
- **Request/Response Headers** e os dados em **Response**

3. Pergunta do Teste

Quantas requisições individuais são feitas para carregar uma única página?

Resposta: Muito mais de uma!



Uma página não é uma requisição

O navegador busca múltiplos recursos de forma independente para compor a tela

1. O Documento Base

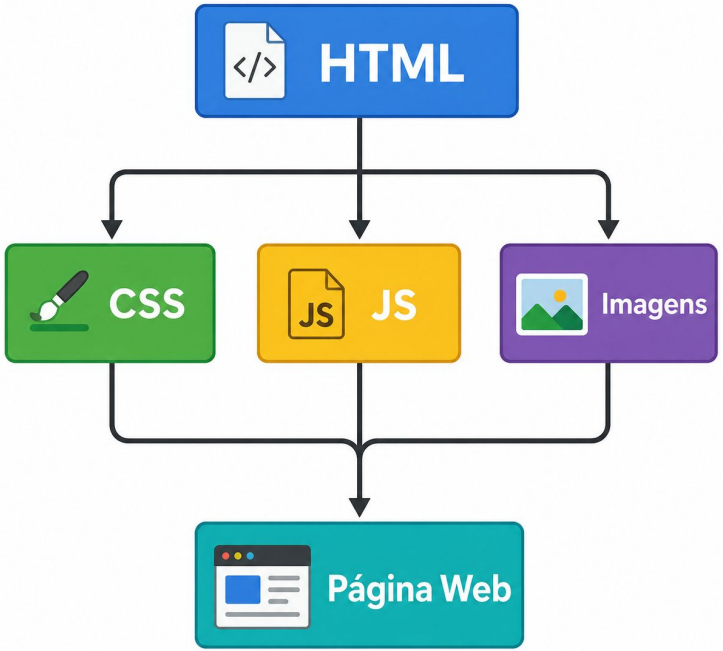
O ponto de partida é o arquivo **HTML** principal, contendo a estrutura da página, mas sem estilos ou mídias embutidas. `<html> → <body>`

2. Requisições Extras

Ao ler o HTML, o navegador descobre e baixa recursos adicionais: **CSS** (estilos), **JS** (comportamento dinâmico) e **Imagens** (mídias).

3. Resultado Final

Após obter e processar todos os elementos, o navegador monta a **Página Web** interativa. `HTML + CSS + JS + IMG`



Por isso, uma única URL pode resultar em dezenas ou centenas de requisições HTTP individuais em segundo plano.

Atividade prática

Vamos investigar de perto o funcionamento real do protocolo HTTP

1. Preparação

Siga as instruções no seu navegador para capturar as requisições de rede:

- Abra o navegador e acesse um site de sua preferência.
- Pressione `F12` para abrir o console de desenvolvedor.
- Navegue até a aba `Network` (Rede).
- Atualize a página (`F5`) para registrar o tráfego.

2. O que Identificar?

Analise os dados capturados e responda:

- ✔ Qual foi a primeira requisição?
- ✔ Qual foi o método HTTP utilizado?
- ✔ Qual foi o status code de retorno?
- ✔ Qual é o Content-Type da resposta?
- ✔ Qual é o tamanho total da resposta?
- ✔ Quantas requisições totais foram feitas?
- ✔ Quais diferentes recursos foram carregados?

Testando HTTP sem navegador

O terminal também pode fazer requisições HTTP usando o comando `curl`

Requisição Simples

```
curl https://example.com
```

- Faz o download do HTML
- Exibe o corpo da resposta diretamente no terminal

Visualizar Headers

```
curl -I https://example.com
```

- Para visualizar os headers
- Ignora o corpo da mensagem e traz apenas metadados

Obter Detalhes

```
curl -v https://example.com
```

- Modo detalhado (verbose)
- Permite observar todo o handshake e protocolo diretamente

Isso permite observar e analisar o funcionamento do protocolo diretamente, sem a abstração do navegador.

HTTP na INF321

E onde entra o Phoenix?

Navegador

Inicia o fluxo enviando uma requisição HTTP para o servidor.

```
GET /users
Host: ufv.br
```

HTTP →

Phoenix Framework

1. **Router** — Direciona a rota
2. **Controller** — Trata a requisição
3. **Context** — Regras de negócio
4. **Ecto** — Interface com o Banco

SQL →

PostgreSQL

Persiste e recupera os dados da aplicação de forma segura.

```
SELECT *
FROM users;
```

O Phoenix recebe e responde às requisições HTTP

Desafio

Analise a requisição HTTP e responda às perguntas propostas

O que acontece aqui?

```
POST /products HTTP/1.1
Host: api.example.com
Content-Type: application/json

{
  "name": "Notebook",
  "price": 5000
}
```

Pergunte aos alunos:

01. Quem está fazendo a requisição?
02. Qual é o método?
03. Qual é o recurso?
04. Qual é o formato dos dados?
05. Para que serve o `Content-Type`?
06. O que o servidor poderia retornar?

E se der errado?

Compreendendo respostas de erro no protocolo HTTP

Imagine esta resposta

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{
  "error": "Product not found"
}
```

Análise do Erro

- A Internet está funcionando? **Sim**
- O servidor respondeu? **Sim**
- A aplicação está 100% correta? **Não**
- O que significa o código 404?
  ↳ **O recurso solicitado não foi encontrado.**

Essa distinção será fundamental para entender erros em aplicações Web.

Resumo

O que aprendemos até aqui?

Uma requisição possui:

- Método
- URL
- Headers
- Body

Uma resposta possui:

- Status Code
- Headers
- Body

Métodos mais importantes:

`GET` = Leitura de dados

`POST` = Criação de dados

`PUT` = Atualização total

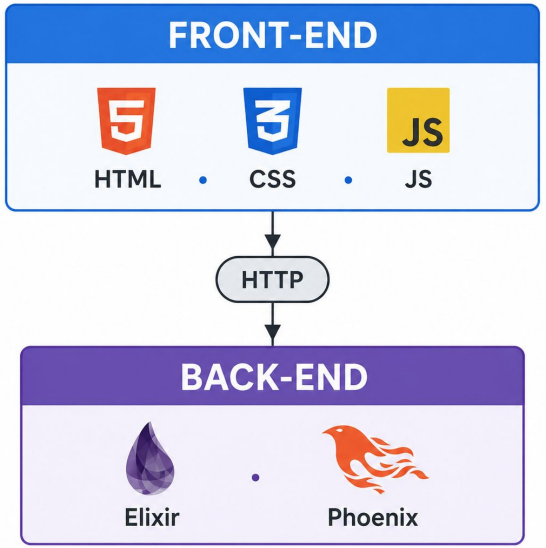
`PATCH` = Atualização parcial

`DELETE` = Remoção de dados

HTTP é o protocolo de comunicação da Web.

Arquitetura de Sistemas Web

Como organizar uma aplicação Web em camadas?



A separação em camadas facilita a manutenção, escalabilidade e evolução independente do sistema.

Por que separar?

Uma aplicação Web possui diferentes responsabilidades

Exemplo: Sistema de Vendas

- Apresentar produtos e receber cliques
- Validar formulários e autenticar usuários
- Calcular preços e processar pedidos
- Armazenar informações com segurança

Colocar tudo no mesmo lugar?

Seria viável ou sustentável construir e manter toda essa lógica misturada em um único arquivo?

Não!

A separação de conceitos permite organizar e evoluir cada responsabilidade de forma independente.

Dividir para conquistar: a separação é a base da Arquitetura em Camadas.

O conceito de camada

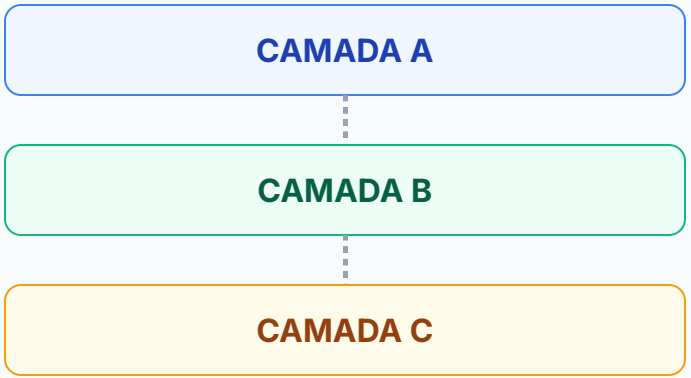
O que é uma camada e como ela se estrutura?

O que é uma camada?

Uma camada é uma parte da aplicação que possui:

- Responsabilidades específicas
- Interfaces bem definidas
- Comunicação com outras camadas

Estrutura Conceitual



Cada camada possui uma função bem definida.

Aplicação Web em duas camadas

Para esta disciplina, vamos começar com duas grandes camadas:

Front-end

(Interface + Interação)

Tudo aquilo que executa e apresenta a interface diretamente ao usuário final.

HTTP

Back-end

(Regras + Dados + Serviços)

Tudo aquilo que executa no servidor, processando a lógica do negócio.

A comunicação entre o Front-end e o Back-end é realizada de forma transparente através do protocolo HTTP.

Front-end

A camada de apresentação e interação da aplicação Web

O que o Front-end faz?

- Interface e apresentação visual dos dados
- Interação direta e engajamento do usuário
- Coleta de dados através de formulários
- Controle de fluxo de navegação
- Comportamento dinâmico da interface

Tecnologias Fundamentais

HTML — Estrutura básica

Define a marcação, os elementos e o esqueleto de conteúdo da página.

CSS — Apresentação visual

Controla o design, as cores, fontes, layouts e a responsividade.

JavaScript — Comportamento

Adiciona interatividade, lógica de cliente e dinamismo à interface.

HTML, CSS e JavaScript trabalham juntos para construir a experiência completa do usuário no navegador.

HTML

A camada de estrutura e conteúdo da página Web

Estrutura do Conteúdo

✦ HTML define **o que existe na página**.

✦ Ele representa de forma semântica a estrutura e toda a base de conteúdo do site.

Regra de Ouro: O HTML carrega o significado, não a estilização visual.

Exemplo Prático

```
<!-- Cabeçalho Principal -->
<h1>Produtos</h1>

<!-- Ação do Usuário -->
<button>Comprar</button>

<!-- Lista de Itens -->
<ul>
  <li>Notebook</li>
  <li>Celular</li>
</ul>
```

O HTML representa a estrutura e o conteúdo fundamentais do documento web.

CSS

A camada de estilo e aparência da página Web



O que o CSS controla?

CSS define **como a interface aparece**. Ele possibilita customizar:

- Cores e fontes
- Espaçamento e tamanhos
- Posicionamento e layout
- Responsividade para dispositivos

Exemplo Prático

```
/* Define estilo do botão */
button {
  background: blue;
  color: white;
  padding: 10px;
}
```

O CSS permite separar totalmente o conteúdo estrutural (HTML) de sua apresentação visual.

JavaScript

A camada de comportamento e interatividade da página Web

Capacidades Dinâmicas

- Responder a eventos e interações
- Alterar a página e o conteúdo (DOM)
- Validar dados de formulários no cliente
- Realizar requisições HTTP assíncronas
- Consumir APIs e serviços externos

Exemplo Prático

```
/* Monitora clique no botão */
button.addEventListener("click", () => {
  alert("Produto comprado!");
});
```

O JavaScript atua diretamente no navegador do cliente para tornar a experiência rica, fluida e totalmente interativa.

Back-end

A camada do servidor, regras de negócio e persistência de dados

O que o Back-end faz?

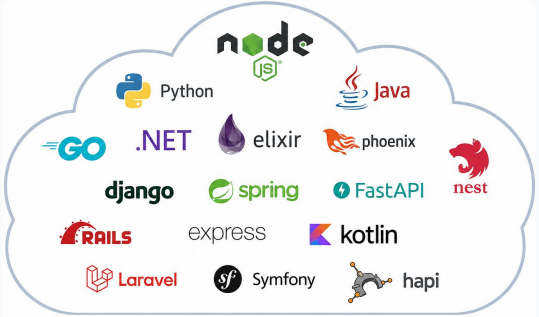
- Executa as regras de negócio
- Processamento e segurança
- Autenticação e autorização
- Conexão com banco de dados
- Disponibilização de APIs

Na nossa disciplina:

Elixir — Funcional
Linguagem dinâmica e funcional projetada para alta escalabilidade.

Phoenix — Framework
Framework Web de alta performance que implementa o padrão MVC.

Mercado & Tecnologias



Elixir e Phoenix formam uma pilha moderna e de altíssima performance para desenvolvimento de APIs e sistemas web.

O banco de dados

A camada de persistência e armazenamento seguro das informações

Fluxo e Controle

✦ **Isolamento do Front-end:** O banco de dados normalmente não pertence e nunca se comunica diretamente com a camada do cliente.

✦ **Intermediação Segura:** O back-end atua como o único ponto de acesso, aplicando regras de negócio e validações necessárias.

✦ **Controle de Acesso:** O back-end gerencia a segurança, autenticação e autorização de todas as consultas aos dados.

Exemplo Prático

PostgreSQL — Sistema Gerenciador

SGBD relacional open-source extremamente robusto e confiável, utilizado na nossa disciplina para persistência confiável dos dados.

```
<!-- Exemplo de Conexão (Ecto/Elixir) -->
defmodule App.Repo do
  use Ecto.Repo, otp_app: :app,
    adapter: Ecto.Adapters.Postgres
end
```

O back-end atua como o único intermediário seguro para realizar consultas e salvar informações no banco de dados.

O fluxo completo

O caminho de ida e volta da informação em uma requisição real

01
USUÁRIO
Ação Inicial
O usuário interage com a interface clicando em "Ver meus produtos".

02
FRONT-END
Requisição
O Front-end envia uma chamada `GET /products` para o servidor.

03
BACK-END
Consulta SQL
O Back-end processa as regras de negócio e consulta o banco.

04
POSTGRESQL
Retorno DB
O SGBD executa a consulta e devolve os dados brutos de forma segura.

05
BACK-END
Resposta JSON
O servidor serializa as informações estruturadas em JSON.

06
FRONT-END
Atualização
O Front-end decodifica os dados recebidos e renderiza na tela do Usuário.

O fluxo de comunicação é síncrono e estruturado, garantindo que cada camada cumpra seu papel com segurança.

HTTP como ponte

A ponte de comunicação essencial entre o Front-end e o Back-end

Fluxo de Comunicação

FRONT-END → [HTTP] → **BACK-END**

✦ A comunicação na Web ocorre estritamente por protocolos.

✦ O cliente envia requisições e o servidor devolve respostas.

Exemplo Prático

```
/* Requisição do Cliente */
GET /products HTTP/1.1

/* Resposta do Servidor */
HTTP/1.1 200 OK
Content-Type: application/json
```

O HTTP funciona como a ponte que conecta o cliente (Front-end) ao servidor (Back-end) através de mensagens estruturadas.

JSON

O formato leve e universal para estruturação e transporte de dados

Exemplo de Estrutura

```
{
  "id": 10,
  "name": "Notebook",
  "price": 5000
}
```

Uma forma muito simples, legível e altamente eficiente de representar informações estruturadas na Web.

Como funciona o transporte?

✦ **Back-end envia:** O servidor processa as informações e as serializa em formato JSON.

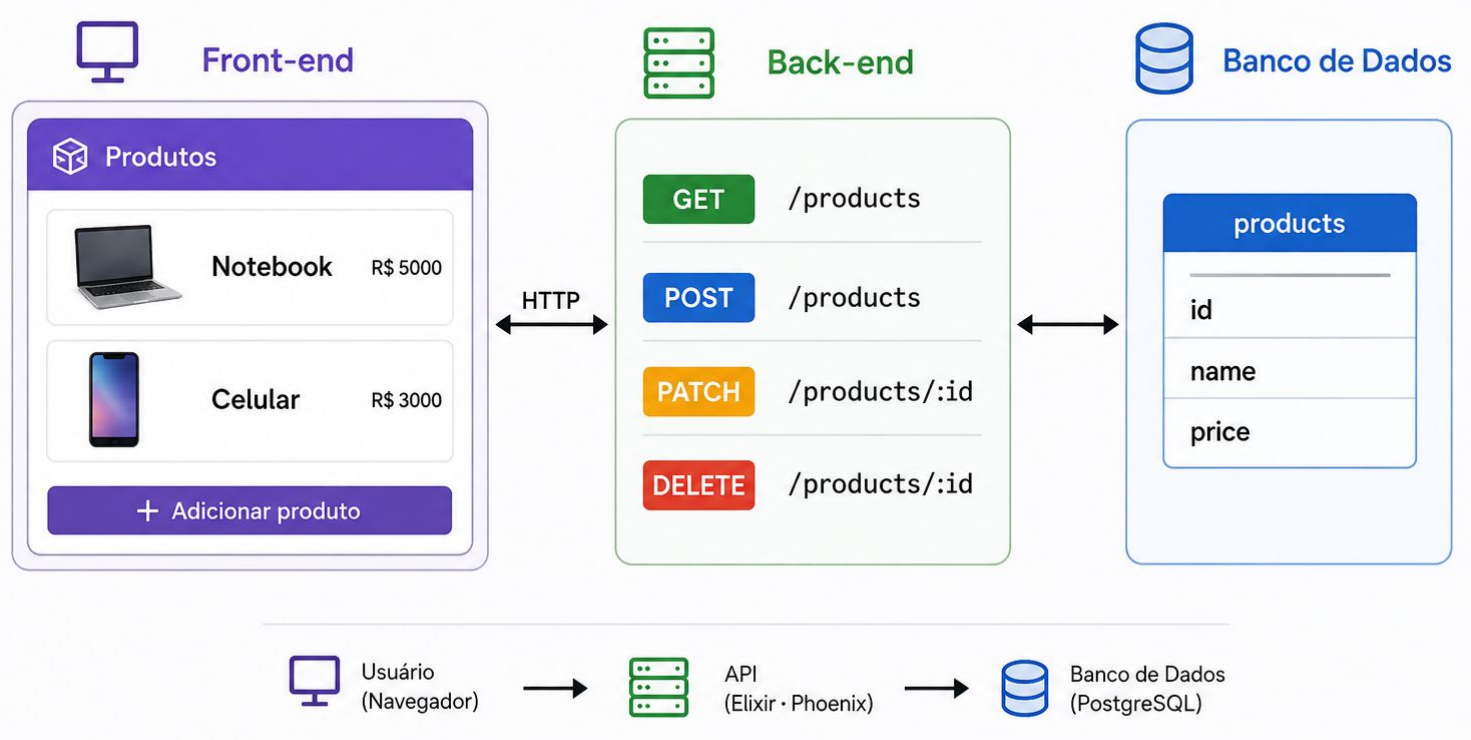
✦ **Front-end interpreta:** O cliente recebe o texto JSON, reconstrói o objeto e o apresenta na tela.

BACK-END➔[JSON]➔FRONT-END

O JSON estabelece um contrato simples e direto de comunicação entre as diferentes camadas da aplicação.

Exemplo Completo

Sistema de Produtos



Separação de responsabilidades

Quem faz o quê?

Responsabilidade	Front-end	Back-end
Interface	✓	
HTML	✓	
CSS	✓	
Interação	✓	
Regras de negócio		✓
Autenticação		✓
Acesso ao banco		✓
API		✓
Persistência		✓

Front-end e Back-end independentes

Podemos trocar uma camada de forma totalmente transparente?

Flexibilidade de Camadas

✦ É possível alterar a tecnologia de uma camada sem impactar a outra.

```
/* Cenário 1: React */
```

React ➔ **Phoenix**

```
/* Cenário 2: Migrando para Vue */
```

Vue ➔ **Phoenix**

Múltiplos Consumidores

✦ Uma única API pode servir diferentes tipos de clientes simultaneamente.

```
/* Cliente Web */
```

Vue ➔ **API Phoenix** ➔ **PostgreSQL**

```
/* Cliente Mobile */
```

Android ➔ **API Phoenix** ➔ **PostgreSQL**

A API funciona como uma interface padronizada de comunicação entre os consumidores e o back-end.

A API como contrato

Como as camadas sabem conversar?

O que é trafegado?

`GET /products`

```
// Retorna:
[
 { "id": 1, "name": "Notebook", "price": 5000 }
]
```

Exemplo prático de requisição e resposta estruturada.

Abstração e Independência

O front-end NÃO precisa saber:

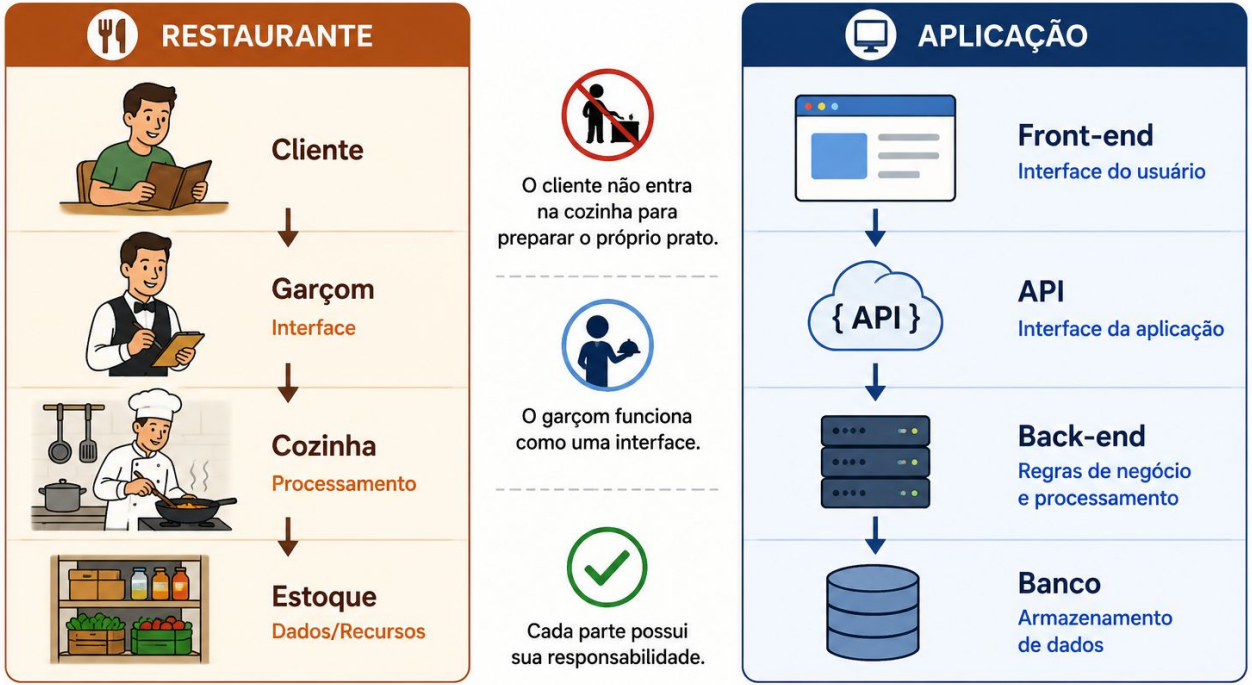
- ✦ Qual banco de dados foi utilizado;
- ✦ Qual linguagem de programação no back-end;
- ✦ Como as consultas internas são realizadas.

✔ **Ele precisa apenas conhecer o contrato da API.**

A API define um contrato simples de comunicação, permitindo que as camadas evoluam de forma totalmente independente.

Uma Analogia

Restaurante vs. Aplicação Web



Assim como em um restaurante, em uma aplicação cada camada tem um papel específico e se comunica com a próxima, garantindo organização e eficiência.

O que acontece se não separarmos?

Os problemas gerados pela falta de organização e alto acoplamento

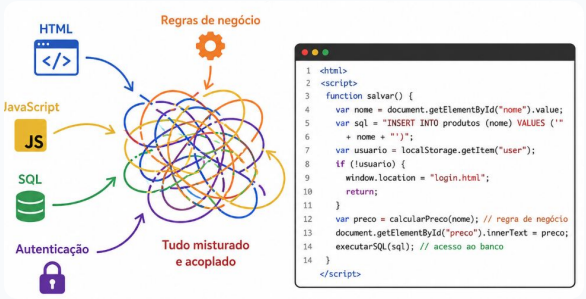
Um sistema sem organização

Tecnologias e responsabilidades misturadas no mesmo arquivo ou camada:

```
HTML + JavaScript (Interface)
+ SQL + Acesso ao banco
+ Regras de negócio
+ Autenticação
```

Tudo misturado e acoplado

Spaghetti Code



O acoplamento extremo gera uma teia complexa de dependências onde alterar uma parte do código pode quebrar todo o sistema de forma imprevisível.

Consequências

- Difícil manutenção
- Difícil de testar
- Difícil reutilização
- Alto acoplamento
- Alto risco de bugs
- Dificulta trabalho em equipe

Benefícios da arquitetura em camadas

Por que utilizar?

Organização

Cada camada possui responsabilidades claras.

Manutenção

Alterações ficam mais isoladas.

Reutilização

Um back-end pode atender vários clientes.

Escalabilidade

As partes podem ser escaladas de forma independente.

Equipe

Diferentes desenvolvedores podem trabalhar em partes distintas.

A arquitetura em camadas promove um desenvolvimento desacoplado, sustentável e altamente eficiente.

Arquitetura da nossa disciplina

O que vamos construir ao longo do semestre?



Uma aplicação web moderna, dividida em responsabilidades claras para facilitar o desenvolvimento e a manutenção.

Discussão

Questões para reflexão sobre a arquitetura web

01. Acesso ao Banco

O front-end pode acessar diretamente o banco de dados?

02. Acoplamento

Por que não? Quais são os principais riscos dessa conexão direta?

03. Independência

O back-end precisa saber qual navegador está sendo utilizado?

04. Multiplataforma

Poderíamos ter um aplicativo Android consumindo o mesmo back-end?

05. Substituição

Poderíamos substituir o front-end sem reescrever o back-end?

06. Protocolo HTTP

Qual é o papel do HTTP nessa arquitetura de comunicação?

Resumo

O que aprendemos até aqui

Front-end

- Interface
- HTML
- CSS
- JavaScript

Back-end

- Regras
- Processamento
- API
- Autenticação
- Dados

Comunicação

- HTTP
- REST
- JSON

Persistência

- PostgreSQL

Uma aplicação Web pode ser organizada em camadas.

Padrão MVC

Model–View–Controller: Organização de software por responsabilidades



Model

- Gerencia os **Dados**
- Aplica as **Regras do Domínio**
- Independe da View



Controller

- **Coordena** o fluxo
- Intermedeia ações
- Atualiza Model e View



View

- Exibe a **Interface**
- Apresenta dados
- Recebe interações

O MVC separa as responsabilidades do sistema, garantindo um código muito mais limpo e fácil de manter.

Model

Representação de informações e comportamentos do domínio da aplicação

Exemplo: Produto

Estrutura de dados do domínio:

`Produto`

- `id`
- `nome`
- `preço`
- `estoque`

O que o Model envolve?

- **Dados:** Estruturas essenciais da entidade.
- **Validações:** Garantia de integridade das informações.
- **Regras de Domínio:** Lógica de negócio específica.
- **Persistência:** Gravação e leitura no banco.

No nosso contexto, o Model estará diretamente relacionado ao Ecto e ao PostgreSQL.

View

Responsável por apresentar informações e interagir diretamente com o usuário

Exemplo de Interface

Produtos

Notebook	**R$ 5.000**
Celular	**R$ 3.000**

Comprar

No Desenvolvimento Web

A View envolve as tecnologias essenciais de front-end:

- **HTML:** Estruturação e semântica do conteúdo.
- **CSS:** Estilização, layout, cores e responsividade.
- **JavaScript:** Comportamento, interações e dinamismo.

A View preocupa-se principalmente com como apresentar as informações.

Controller

Recebe uma requisição e coordena o processamento de forma centralizada.

Coordenação Geral

Ponto de Entrada: Funciona como a porta de entrada para todas as requisições da aplicação.

Exemplo de Rota: Processa requisições como `GET /products`.

Intermediação: Direciona o fluxo entre o Model e a resposta final.

Fluxo de Execução

1. **Receber:** Recebe a requisição externa.
2. **Solicitar:** Pede dados e lógica de negócio ao Model.
3. **Processar:** Obtém o resultado gerado pelo Model.
4. **Responder:** Prepara e envia a resposta (View / JSON).

O Controller centraliza a coordenação da aplicação, agindo como o ponto de controle central.

Exemplo de Fluxo

Acompanhe o caminho de uma requisição no padrão MVC para a listagem de produtos

1. Requisição

O usuário acessa a página e dispara:

`GET /products`

2. Controller

Recebe a rota solicitada e coordena o fluxo, acionando o Model.

3. Model

Busca os dados diretamente no banco:

PostgreSQL

E retorna a lista de produtos estruturada.

4. View

Apresenta as informações renderizadas:

- **Notebook:** R$ 5.000
- **Celular:** R$ 3.000

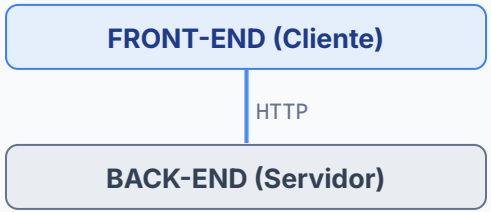
O fluxo demonstra a separação clara: o Controller coordena, o Model obtém os dados e a View os exibe.

MVC não é a arquitetura inteira

Uma distinção importante de níveis de abstração

Cliente e Servidor

Representa uma separação ampla e física da infraestrutura web.



Organização Interna

O MVC organiza as responsabilidades e o código interno da própria aplicação.

1. Arquitetura Web Geral
2. Divisão Front-end ↔ Back-end
3. **MVC (Responsabilidades Internas)**

São níveis diferentes de abstração. MVC organiza responsabilidades dentro de um dos lados.

E quando temos uma API REST?

MVC também pode aparecer em APIs

Exemplo de Retorno

Uma API não precisa necessariamente retornar HTML. Por exemplo, a rota:

```
GET /products

[
  {
    "id": 1,
    "name": "Notebook",
    "price": 5000
  }
]
```

Adaptação do Fluxo MVC

1. Controller
Intercede na requisição recebida (GET) e delega o processamento dos dados.

2. Model / Context
Busca as informações requisitadas no banco de dados e retorna os objetos de domínio.

3. JSON (A "View")
Atua na formatação final dos dados para entrega direta ao cliente, sem necessidade de HTML.

A "View" pode ser entendida de forma mais ampla como a representação dos dados, ou a camada responsável por formatar a resposta.

MVC × Front-end × Back-end

Não confunda os conceitos

Arquitetura Web

Trata da separação física e de comunicação entre cliente e servidor.

FRONT-END (Cliente)

▲ HTTP ▼

BACK-END (Servidor)

Organização Interna (MVC)

Organiza as responsabilidades de código dentro do próprio servidor (Back-end).

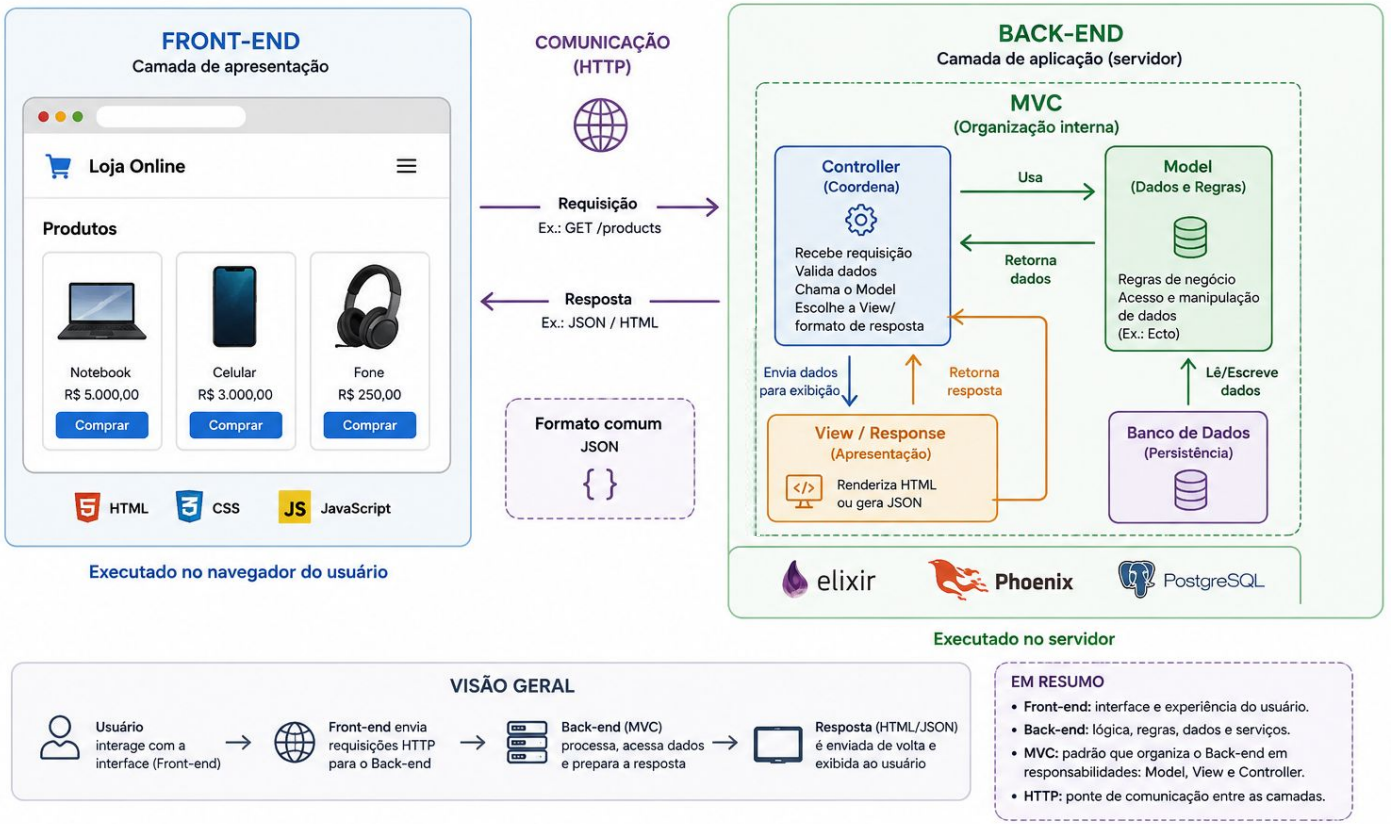
- **Controller** → Recebe requisição e delega fluxo
- **Model / Domain** → Lógica de negócios e dados
- **Database** → Banco de dados

Apresentação: Pode ser via **View** (HTML) ou **JSON** (Front-end separado).

A Arquitetura Web separa fisicamente cliente e servidor, enquanto o MVC organiza a lógica interna dentro do Back-end.

Front-end, Back-end e MVC

Como convivem em um sistema Web



Atividade rápida

Quem é quem? • Classifique os elementos no padrão MVC

Considere o Cenário

Leia atentamente a execução abaixo:

"Um usuário acessa `/products`. O servidor busca os produtos no **PostgreSQL** e retorna uma página contendo uma lista de produtos."

Classifique cada Elemento

Elemento	MVC
`/products`	?
Receber a requisição	?
Buscar produtos	?
PostgreSQL	?
HTML da lista	?
Apresentar os produtos	?

Pergunta final: **Qual seria a vantagem de não colocar toda essa lógica em um único arquivo?**

Atividade rápida

Quem é quem? • Classifique os elementos no padrão MVC

Elemento	MVC	Por quê?
`/products`	**Controller**	A rota direciona a requisição para o componente que irá tratá-la.
Receber a requisição	**Controller**	O Controller recebe e processa a requisição HTTP.
Buscar produtos	**Model**	O Model é responsável pelo acesso/manipulação dos dados.
PostgreSQL	**Model / Persistência**	O banco armazena os dados; o Model faz a mediação entre aplicação e banco.
HTML da lista	**View**	O HTML representa os dados para apresentação.
Apresentar os produtos	**View**	A View é responsável pela apresentação das informações ao usuário.

Pergunta final: **Qual seria a vantagem de não colocar toda essa lógica em um único arquivo?**
Resposta: Separação de responsabilidades.

Conclusões

Introdução à Web • Resumo dos conceitos fundamentais

Infraestrutura & Rede

Internet
Infraestrutura global de comunicação que conecta dispositivos em todo o mundo.

Web (World Wide Web)
Serviço de informação baseado na Internet que funciona sob o modelo cliente-servidor.

HTTP
Protocolo de comunicação que define como as mensagens são formatadas e transmitidas.

Desenvolvimento & Arquitetura

Front-end
Interface gráfica do usuário e interação direta no navegador.

Back-end
Processamento lógico, regras de negócio e serviços no servidor.

MVC
Padrão de arquitetura para organizar as responsabilidades internas da aplicação.

Banco de Dados
Persistência e integridade das informações do sistema.

A compreensão de infraestrutura, arquitetura (MVC) e persistência forma a base para construir os sistemas Web modernos.



Bons estudos!

Prof. Lucas Vegi